

ОТЧЕТ ПО ПРОГРАММНОЙ РЕАЛИЗАЦИИ

ЗАДАЧИ: «РАСПРЕДЕЛЕНИЕ ГОРОДОВ МЕЖДУ ГОСУДАРСТВАМИ»

Выполнил: Тотмянин Данил Денисович

Группа: 23Б10

1. Постановка задачи

Имеется множество городов и дороги, связывающие эти города. Для каждой дороги задана её длина. Граф дорог является связным. Заданы k столиц, формирующих центры будущих государств.

Цель задачи — распределить все города между государствами согласно следующему алгоритму:

1. По очереди каждому государству добавляется ближайший незанятый город, непосредственно связанный дорогой с любым городом, уже принадлежащим этому государству (столицей или городом, добавленным на предыдущих шагах).
2. Ближайшим считается город, расстояние от которого до границ государства (длина конкретного ребра) минимально.
3. Процесс продолжается циклично, пока все города графа не будут распределены.

Входные данные:

- N — число городов, M — число дорог.
- M строк с описанием дорог: `u v len` (откуда, куда, длина).
- K — количество столиц, за которым следует K номеров городов-столиц.

2. Описание алгоритма и структур данных

В основе программы лежит **жадный алгоритм поиска на графах от нескольких источников** (похожий на одновременный запуск алгоритма Прима из нескольких стартовых вершин).

Структуры данных:

- **Граф дорог:** хранится в виде списка смежности `std::vector<std::vector<GraphNode>>`.
- **Массив принадлежности (`city_to_state`):** Вектор, где индекс — это номер города, а значение — номер государства (или `-1`, если город свободен).
- **Приоритетные очереди (Min-Heap):** Для каждого государства создается своя очередь `std::priority_queue`, сортирующая доступные для захвата соседние города по возрастанию длины дороги (`weight`).

Логика работы:

1. Инициализируются k государств. В их очереди помещаются все города, смежные со столицами.

2. Запускается циклический обход (Round-Robin). В свой ход государство извлекает из очереди город с минимальным весом ребра.
3. Если извлеченный город еще не занят другим государством, он присоединяется к текущему. Все свободные соседи этого нового города добавляются в очередь государства.
4. Цикл завершается, когда счетчик распределенных городов достигает N .

3. Программная реализация (GUI)

- **Язык программирования:** C++ (стандарт C++17).
- **Фреймворк:** Qt 6 (графический интерфейс).
- **Особенности интерфейса:** Интерфейс выполнен в светлой теме с разделением на панели (QSplitter).
 - **Левая панель:** Текстовое поле для ввода данных структуры графа и вывода подробного текстового лога (списки городов по государствам).
 - **Правая панель (2 вкладки):**
 1. *Интерактивное полотно:* Круговая укладка графа. Узлы окрашены в пастельные цвета государств, столицы выделены жирной обводкой. Веса ребер подписаны красным шрифтом на белом фоне.
 2. *Матрица смежности:* Таблица, где строки и столбцы окрашены в цвета государств. Ячейки пересечений внутри одного государства подсвечиваются полупрозрачным фоном.

4. Тестирование

Для проверки корректности алгоритма были проведены три тестирования с различной топологией графа.

Тест 1: Базовый сценарий (Равномерное распределение)

Цель теста — проверить корректность работы на простом графе с двумя центрами, расположенными на разных краях.

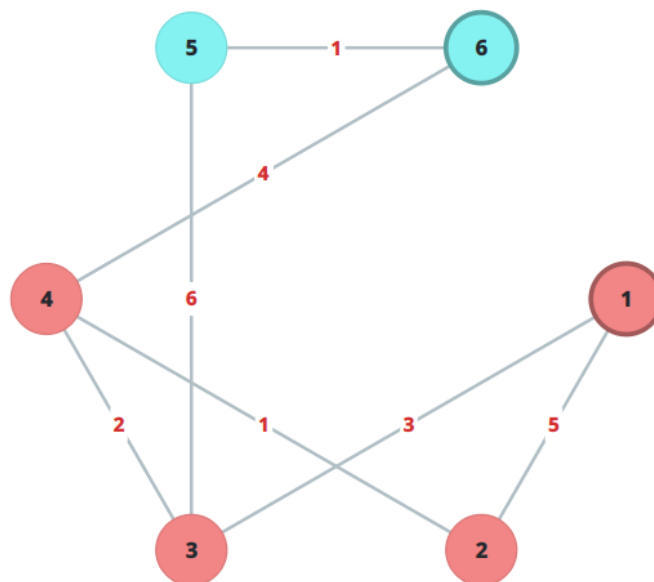
Входные данные:

```
6 7
1 2 5.0
1 3 3.0
2 4 1.0
3 4 2.0
3 5 6.0
4 6 4.0
5 6 1.0
2
1 6
```

Разбор выполнения:

- **Ход 1:**

- Гос-во 1 (Ст. 1) видит города 2 (вес 5) и 3 (вес 3). Забирает **3**.
- Гос-во 2 (Ст. 6) видит города 4 (вес 4) и 5 (вес 1). Забирает **5**.
- **Ход 2:**
 - Гос-во 1 ищет минимум среди оставшихся от г. 1 (г.2, вес 5) и новых от г. 3 (г.4, вес 2; г.5 - уже занят). Забирает **4**.
 - Гос-во 2 ищет минимум среди оставшихся. Единственный доступный свободный — г.2 от г.5 (нет связи) или от г.6 (через г.4 - занят). Гос-во 2 пропускает ход, так как пограничные города кончились.
- **Ход 3:**
 - Гос-во 1 имеет доступ к г.2 (вес 1 от г.4). Забирает **2**.



	Г1	Г2	Г3	Г4	Г5	Г6
Г1	0	5	3	-	-	-
Г2	5	0	-	1	-	-
Г3	3	-	0	2	6	-
Г4	-	1	2	0	-	4
Г5	-	-	6	-	0	1
Г6	-	-	-	4	1	0

Тест 2: Конкуренция за центральный узел (Гонка)

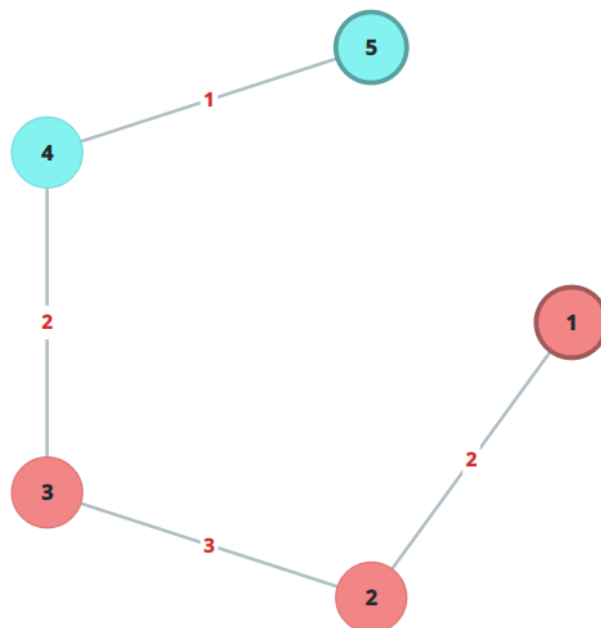
Цель теста — продемонстрировать, как очередность хода влияет на распределение, когда два государства претендуют на один город.

Входные данные:

```
5 4
1 2 2.0
2 3 3.0
5 4 1.0
4 3 2.0
2
1 5
```

Разбор выполнения:

- **Ход 1:** Гос-во 1 забирает город **2** (вес 2.0). Гос-во 2 забирает город **4** (вес 1.0).
- **Ход 2:** Гос-во 1 видит город 3 (с весом 3.0 от г.2). Гос-во 2 тоже видит город 3 (с весом 2.0 от г.4). Несмотря на то, что у Гос-ва 2 дорога к г.3 короче, сейчас **ход Гос-ва 1**, и город 3 — его единственный и лучший вариант. Гос-во 1 забирает **3**.
- **Итог:** Гос-во 1 (1, 2, 3), Гос-во 2 (5, 4). Алгоритм строго соблюдает правило поочередности ходов.



	Г1	Г2	Г3	Г4	Г5
Г1	0	2	-	-	-
Г2	2	0	3	-	-
Г3	-	3	0	2	-
Г4	-	-	2	0	1
Г5	-	-	-	1	0

Тест 3: Захват территорий по короткой цепочке

Цель теста — проверить ситуацию, когда одно государство имеет длинные дороги от столицы, а второе — короткие, что позволяет второму «внедриться» в территорию первого.

Входные данные:

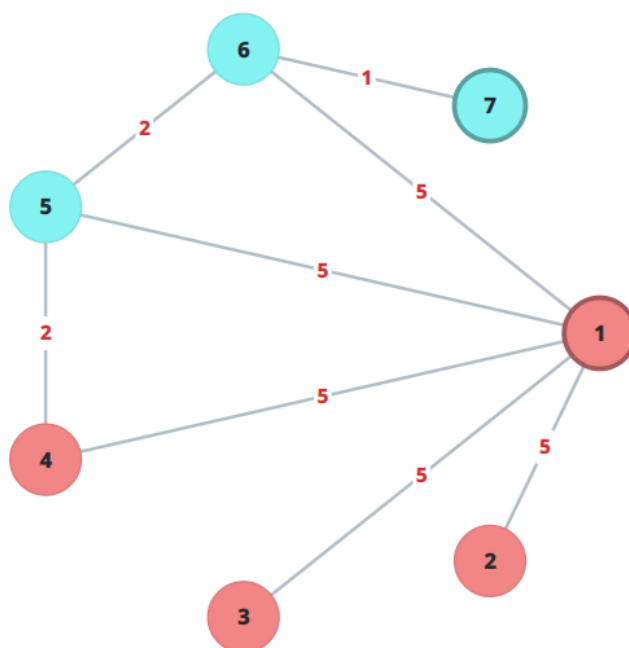
```

7 8
1 2 5.0
1 3 5.0
1 4 5.0
1 5 5.0
1 6 5.0
7 6 1.0
6 5 2.0
5 4 2.0
2
1 7

```

Разбор выполнения:

В то время как Гос-во 1 медленно забирает вершины звезды за счет длинных ребер (5.0), Гос-во 2 (столица 7) быстро забирает город 6 (вес 1.0), затем от него забирает город 5 (вес 2.0), а затем город 4 (вес 2.0). За счет малых весов локальных дорог Гос-во 2 успевает отрезать часть «звезды» у Гос-ва 1.



	Г1	Г2	Г3	Г4	Г5	Г6	Г7
Г1	0	5	5	5	5	5	-
Г2	5	0	-	-	-	-	-
Г3	5	-	0	-	-	-	-
Г4	5	-	-	0	2	-	-
Г5	5	-	-	2	0	2	-
Г6	5	-	-	-	2	0	1
Г7	-	-	-	-	-	1	0

5. Заключение

Разработанное программное обеспечение на языке C++ с использованием Qt полностью решает задачу маршрутизации и кластеризации городов.

Алгоритм успешно справляется с поочередным жадным распределением графа от нескольких источников (столиц).

Светлый графический интерфейс с двумя типами визуализации (круговой граф с выделением столиц и подкрашенная матрица смежности) позволяет наглядно анализировать и проверять логику принадлежности каждого города к конкретному государству. Программа устойчива и отрабатывает корректно даже в случаях конкуренции государств за одни и те же пограничные узлы.